Prayog — Failure and Recovery Test Report

CS455 Deliverable 3 · Group 3 · 6 November 2026

1. Summary

Four live fault-injection drills were run against a complete system — API (uvicorn), scheduler tick, a simulated node agent with 4 GPU units, and PostgreSQL 16 — on a freshly migrated and seeded database. Every component was killed hard (`taskkill /F`, no graceful shutdown) while work was in flight. The automated failure-path tests already in the suites were run as well (section 6).

Drill	Fault	Detected	Recovered	Data lost	Verdict
F1	node agent killed while a 2-GPU job runs	node `DOWN` after **8.9 s** (timeout 8 s + 2 s tick)	job requeued as `NODE_FAILURE` at once; running again **14.8 s** after the kill, on a new allocation	none	**pass**
F2	API process killed	clients get `ConnectError`	API back in **9.8 s**; a retry with the same `client_request_id` returned the **same job** (200), 1 row in the table; the old token still valid	none	**pass**, but see F2-side effect (section 5)
F3	PostgreSQL stopped for 10.9 s under a running API	requests answered **500** after ≈3 s each; API process stayed up	first request after restart succeeded **without restarting the API** (`pool_pre_ping`)	none	**pass with defect D3-06** (500, not 503) — fixed, re-run in section 4
F4	scheduler tick killed; then the agent killed too	—	a new job was still placed (event-driven pass on submit); **but the dead node stayed `AVAILABLE` for 14 s** until the tick was restarted, then `DOWN` within 6.6 s	none	**pass, with an operational requirement**: the tick must be supervised
side effect of F2 and F3	API or DB outage longer than the heartbeat timeout	—	—	running jobs evicted	**defect D3-08, now fixed (section 5)**

Configuration for the drills (to make them fast; production defaults in brackets): `PRAYOG_SCHED_HEARTBEAT_TIMEOUT_SECONDS=8` [30], `HEARTBEAT_INTERVAL_SECONDS=2` [10], `SCHEDULER_TICK_SECONDS=2` [10]; agent `--interval 1 --run-seconds 120`.

2. Environment and procedure

- Code: `main` at `18767aa` (F1–F4), and branch `d3-testing` for the re-run in section 4.
- Script: `evidence/failure_drills.py` — creates database `prayog_d3_fail`, runs `alembic upgrade head` and `seed_dev`, starts the four processes, injects each fault, polls the database every 250 ms and writes a timestamped timeline (`evidence/failure_drills.json`). After the run the allocation history and the `node_alerts` table were read back to check what happened in between.

3. Timeline (seconds since start)

t (s)	Event
20.7	system up; 23.8 node `n1` AVAILABLE
26.5	**F1** job 1 (2 GPUs) RUNNING
26.8	agent killed
35.7	node DOWN (detect 8.86 s); job 1 QUEUED, allocation released `NODE_FAILURE`
36.2	agent restarted with the same state directory (stored credential, no token)
40.6	node AVAILABLE (4.3 s after restart)

t (s)	Event
41.7	job 1 RUNNING again on allocation #2 — total recovery 14.8 s
42.7	**F2** API killed; a submit during the outage fails with `ConnectError`
52.6	API back (downtime 9.8 s); retry of `client_request_id=f2-retry` → 200, same request ID, one row
57.6	**F3** PostgreSQL stopped; three requests → 500, 500, 500 (3.1 s, 2.8 s, 2.8 s); API process alive
68.5	PostgreSQL started (down 10.9 s); 69.4 API serving again, same process
70.2	**F4** tick killed; 71.4 new job 3 SCHEDULED without the tick
71.4–85.9	agent killed; node stays AVAILABLE with no reaper
92.5	tick restarted → node DOWN 20.8 s after the agent died
96.5	agent back, node AVAILABLE

4. Re-run of F3 after fix D3-06

The full drill was run again on branch `d3-testing` (commit `f358e01`), same configuration, fresh database (`evidence/failure_drills_after.json`):

Drill	Before the fix (`main`)	After the fix
F3, three requests while PostgreSQL is down	`500`, `500`, `500` (3.1 s, 2.8 s, 2.8 s)	`503, 503, 503` (2.9 s, 3.0 s, 2.9 s); the `Retry-After: 5` header is checked by `test_database_unavailable`
F3, recovery	served again 0.9 s after the database returned, same API process	served again 0.8 s after the database returned, same API process
F1 node loss → running again	14.8 s	15.0 s
F2 API restart, retried submission	same request ID, one row	same request ID, one row
F4 tick down, then restarted	node DOWN 20.8 s after the agent died	node DOWN 20.9 s after the agent died
`NODE_DOWN` alerts for 2 real agent kills	4	3 — **D3-08 reproduces** (it is not addressed by these fixes)

The ≈ 3 s per request during the outage is the driver's connection attempt (`pool_pre_ping`), not the pool timeout.

5. Defect found and fixed: a control-plane outage is mistaken for node failure (D3-08)

The drill killed the node agent twice (F1, F4) but the `node_alerts` table showed **four** `NODE_DOWN` alerts, and the allocation history showed the running jobs released as `NODE_FAILURE` four times. Allocations #2 to #5 were lost during the API and database outages, on a healthy agent.

Impact. Any API restart, deployment or database fail-over longer than about 30–40 s marked every node DOWN and requeued every running job, wasting all GPU-hours in flight.

Root cause. The reaper compared each node's `last_heartbeat_at` with the clock; it could not tell "the node stopped" from "the platform stopped listening".

Fix. Each API process stamps `platform_liveness` every 2 s (`logic/gpu/scheduler/liveness.py`). The reaper judges heartbeats and dispatch timeouts only when the stamp is fresh and the API has been up for a full heartbeat timeout since its last gap. A startup check refuses a configuration where the stamp is too slow to show an outage before healthy nodes look dead.

Measured at production timeouts (30 s heartbeat timeout, 10 s heartbeat, 10 s tick; `backend/experiments/failure_experiment.py`), with the tick left running:

API stopped for 45 s	Before the fix	After the fix
Node during the outage	DOWN	**AVAILABLE**
Job during the outage	QUEUED	**RUNNING**
Job's original allocation still live afterwards	False	**True**
False `NODE_DOWN` alerts	1	**0**

A node that really dies is still caught: 33.9 s after the agent stopped, inside the 40 s bound.

Regression tests. `test_cases/gpu/scheduler/test_liveness.py`: outages at both layers, the full timeout after return, a real death still detected, an outage using up no dispatch attempt, and the configuration check.

5.1 Failure scenarios at production timeouts, final build

Scenario	Fault	Result
Database outage	PostgreSQL stopped for 60 s with a job running	Every submit answered `503` with `Retry-After`; the tick survived; healthy again 0.3 s after the database returned; the retried request made 1 row; the job stayed `RUNNING`, the node `AVAILABLE`
API crash	`docker kill` with jobs RUNNING, SCHEDULED and QUEUED	Healthy in 2.7 s after restart; states unchanged: True; job count unchanged: True; the scheduled job started 5.5 s later
Node death	Agent stops mid-run	DOWN in 33.9 s; job requeued; submit time preserved: True; charged usage rows: 0; alert raised: True; running on a spare node 0.9 s later
Duplicate transactions	The same submit twice, an acknowledgement three times, a report three times	Submit answers [201, 200], 1 row; acknowledgements [200, 200, 200], 1 run; 1 usage row
Dispatch timeout	Agent never acknowledges	Allocation released after 64.1 s (setting 60 s) and the job re-placed
Tick crash	`docker kill` on the tick	A new job was still placed in 0.2 s by the event-driven pass; the tick restarted

All invariants held after every scenario. Raw results: `evidence/failure_before_outages.json`, `evidence/failure_after.json`.

A limit worth knowing. The first version of this experiment did not reproduce D3-08: an agent heartbeating every 2 s reached the server before the next tick, and a 7 s crash was under the timeout. So a passing result here only means something at production timings, with an outage longer than the timeout.

LLM failure. `test_a_release_hook_that_raises_does_not_stop_the_release` shows a failing Steward hook cannot hold GPUs. The Steward Runtime being down is covered by the Steward suite (table below).

6. Automated failure-path tests

Failure	Test(s)	Result (6 Nov)
Heartbeat lost → node DOWN, jobs requeued with original submit time	`test_gpu_agent_api`, `test_hardware_investigation`	pass
Dispatch not acknowledged → retried, FAILED after max attempts	`test_gpu_agent_api`	pass

Failure	Test(s)	Result (6 Nov)
Agent restarts mid-run, never runs a job twice	`test_node_agent_runs::test_the_workspace_is_removed_and_nothing_is_run_twice`	pass on Linux (CI); Windows-only failure (`killpg`)
Walltime exceeded → stopped, checkpoints kept	`test_node_agent_runs::test_the_agent_stops_a_run_at_its_walltime_and_keeps_its_checkpoints`	pass on Linux (CI); Windows-only failure
Missing or corrupt artifact → snapshot INCOMPLETE, no rebuild from it	`test_mlflow_snapshot_storage_failures` (4), `test_mlflow_lifecycle`	pass
Steward Runtime down → runs still seal, jobs still schedule	Steward integration (PRAYOG-102)	pass
Outbox delivered twice → consumed once	`test_mlflow_outbox` (10), `test_mlflow_ledger_consumer` (5)	pass
Database unreachable → 503 + Retry-After	`test_database_unavailable` (2, new)	pass
Inconsistent unit row → pass continues on other nodes	`test_concurrency::test_c6_*` (new)	pass
API or database outage is not a node failure	`test_cases/gpu/scheduler/test_liveness.py` (9, new)	pass

7. Recovery objectives observed

Component	Observed recovery	Target (Deployment section 7)
Node agent crash	14.8 s to the job running again (drill timeouts); ≈ 45 s with production timeouts	job back in the queue within `heartbeat_timeout + tick`
API crash	9.8 s (manual restart in the drill; `Restart=always` in systemd)	< 30 s for users; running jobs are no longer affected (D3-08 fixed)
Database outage	immediate after the database returns, no restart	RTO ≤ 1 h for a lost database (restore from dump + WAL) — not drilled
Scheduler tick	must be supervised; nodes are not reaped while it is down	systemd `Restart=always`